

COMPARE3

비교 — 트랜스포머 vs 선형(naive/chunked 병렬) vs 선형(재귀, 메모리캐싱)

같은 차원(d256/h8/L4)에서 시퀀스 길이별 peak GPU 메모리. 4090 실측. 재현: `compare3.py`.

경위: 처음엔 우리 선형 '병렬'이 $L \times L$ 행렬을 materialize하는 $\text{naive}(O(L^2))$ 구현이라 16384에서 OOM 났다. 이는 선형어텐션 장점을 버린 구현 결함 → chunked (GLA 표준, $O(L)$)로 재구현해 해결.

peak 메모리 (forward, batch=1)

L	TF(full, FlashAttn)	선형(naive)	선형(chunked)	선형(재귀)	TF KV캐시(추론)	재귀 상태
4096	182 MB	2309 MB	170 MB	170 MB	32 MB	132 KB
8192	245 MB	8827 MB	220 MB	220 MB	64 MB	132 KB
16384	371 MB	OOM	321 MB	—	128 MB	132 KB
32768	623 MB	OOM	522 MB	—	256 MB	132 KB

정확도 (동치)

naive == chunked == 재귀: 모두 같은 가중치, 출력 diff $\sim 5e-7$ (verify_recurrent.py + chunked 검증). 손실 0. 트랜스포머는 다른 아키텍처(회상은 softmax가 더 강함, GROKING.md).

결론 (정정 후 — 정직)

항목	결과
naive 병렬 OOM	✗ 우리 구현 결함이었음($O(L^2)$). chunked로 해결
학습/prefill 메모리	chunked 선형 $O(L)$ — 긴 길이에선 FlashAttn 트랜스포머보다도 적음(16384: 321 vs 371MB, 32768: 522 vs 623MB)
추론 메모리	재귀 상태 $O(1)=132KB$ 상수 vs 트랜스포머 KV캐시 $O(L)$ (16384=128MB) → $\sim 992\times$, 길이 무관
정확도	naive=chunked=재귀 동치($5e-7$)

→ 제대로 구현하면(chunked) 선형/Titans는 학습 메모리는 트랜스포머와 동급 이상, 추론 메모리는 압도적(상수, $\sim 992\times$). 사용자가 지적한 OOM은 우리 naive 구현 버그였고 수정됨.

연산(시간) — FlashAttention은 메모리만 선형, 연산은 여전히 $O(L^2)$

"FlashAttn도 선형인데 우리도 선형이면 의미 없지 않나?"에 대한 답. FlashAttention은 $L \times L$ 를 HBM에 materialize하지 않을 뿐 연산량(FLOPs)은 $O(L^2)$ 그대로다. 선형/chunked는 연산도 $O(L)$.

forward 시간 vs L (4090, 동일 d256/h8/L4, batch1):

L	TF(flash)	선형(chunk)	TF/Lin
2048	2.0 ms	14.9 ms	0.14
8192	14.9 ms	55.0 ms	0.27
32768	174.5 ms	213.6 ms	0.82
65536	660.8 ms	426.5 ms	1.55 (선형 승)
131072	2577 ms	865 ms	2.98

$2 \times L$ 일 때 TF $\sim 4 \times (=O(L^2))$, 선형 $\sim 2 \times (=O(L))$. 교차점 $\sim 50K$, 이후 격차 확대.

	메모리	연산
FlashAttention	$O(L)$	$O(L^2)$
선형/chunked	$O(L)$	$O(L)$

→ 선형의 의미: ① 긴 컨텍스트 연산 $O(L)$ (실측 128K 3× 빠름), ② 추론 상태 $O(1)$ ($\sim 992 \times$), ③ 추론 토큰당 $O(1)$. FlashAttn이 못 주는 부분. 재현: `timebench.py`.

검증된 알고리즘(fla deltanet)으로 재확인

위 표는 from-scratch 선형. 검증된 fla DeltaNet vs 트랜스포머(flash) 재측정(`efficiency_bench.py` , bf16): 131072에서 DeltaNet 31ms/1.15GB vs TF 244ms/1.17GB → 시간 $7.8 \times (O(L) \text{ vs } O(L^2))$, 메모리 동급. 효율 주장이 정확 구현으로 확정. 상세는 [report/REPORT.md](#) §2.

재현

```
python3 tutorial/autoresearch/compare3.py # 메모리 → compare3_result.json
python3 tutorial/autoresearch/timebench.py # 시간 vs L
~/gm_venv/bin/python packages/growing-memory/efficiency_bench.py # deltanet vs flash(bf16)
```